

CANDY DIALOGUE ENGINE

Installation

1.1.0

This document explains how to install Candy Dialogue Engine in a Godot project for the first time.

To update Candy Dialogue Engine inside a project, see **Update_Instructions.pdf** instead.

WARNING:

Make a backup of your project before you continue!

Overview:

If you are installing Candy Dialogue Engine for the first time in a project, you only need to do the following:

- Copy the contents of the 'First_Install' folder to the 'Candy_DE' folder. Merge when prompted.
- Copy 'Candy_DE' to your project's root folder.
- Add 2 autoloads:
 - Candy_DE/Scripts/User_Data/Candy_Functions.gd → name 'candy_de'.
 - Candy_DE/Scenes/Candy_UI.tscn → name 'candy_ui'.
- Add 1 input map action (+3 optional actions):
 - Dialogue_Advance (required)
 - Dialogue_Skip (optional)
 - Dialogue_Speed (optional)
 - Dialogue_Slow (optional)
- Project → Tools → "Upgrade project files..." to fix UID errors.
- Done.

Read below for detailed instructions.

Read even further below for important information after installing Candy Dialogue Engine.

Detailed Instructions:

STEP 0 - Basics

Normally, if you are reading this guide, you have already done this:

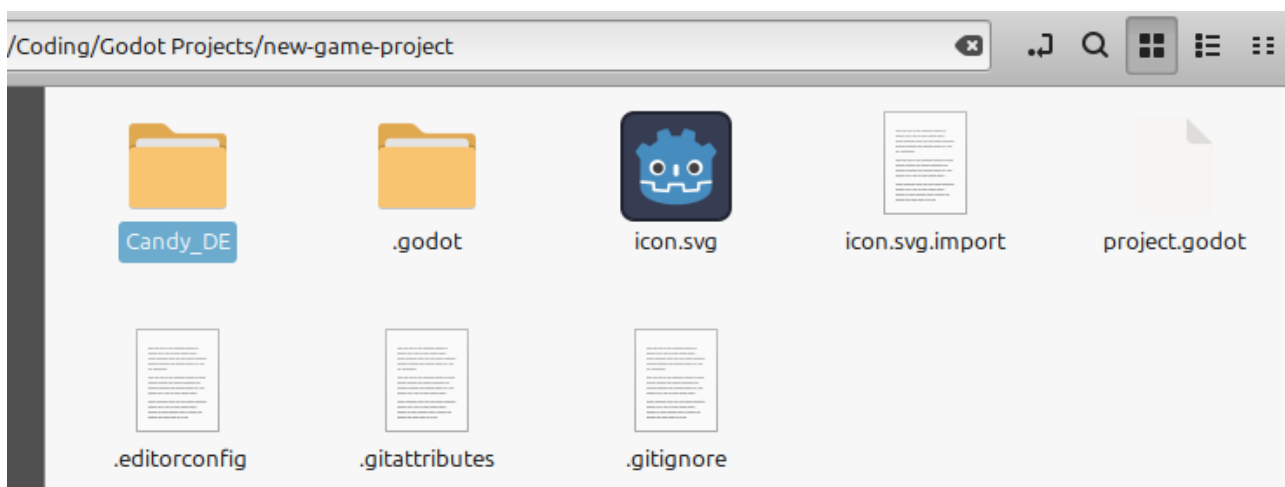
1. Download Candy Dialogue Engine from GitHub: <https://github.com/Candy-Arts/Candy-Dialogue-Engine/releases>
 2. Unpack (unzip) the downloaded .zip file
 3. Open the folder that was just unpacked: **Candy-Dialogue-Engine-[version]**
-

STEP 1 - Merge folders

Never do this when you are updating an existing installation of Candy Dialogue Engine inside a Godot project! The 'First_Install' folder is only for fresh installations, not updates!

1. In the unpacked **Candy-Dialogue-Engine-[version]** folder, open the 'First_Install' folder.
 2. Copy the 'Scenes', 'Scripts', 'Media' and 'Dialogues' folders.
 3. Go back to **Candy-Dialogue-Engine-[version]**.
 4. Open the 'Candy_DE' folder.
 5. Paste the folders you copied from 'First_Install' inside 'Candy_DE'.
 6. Agree to merge any folders when prompted.
-

STEP 2 - Copy Candy_DE



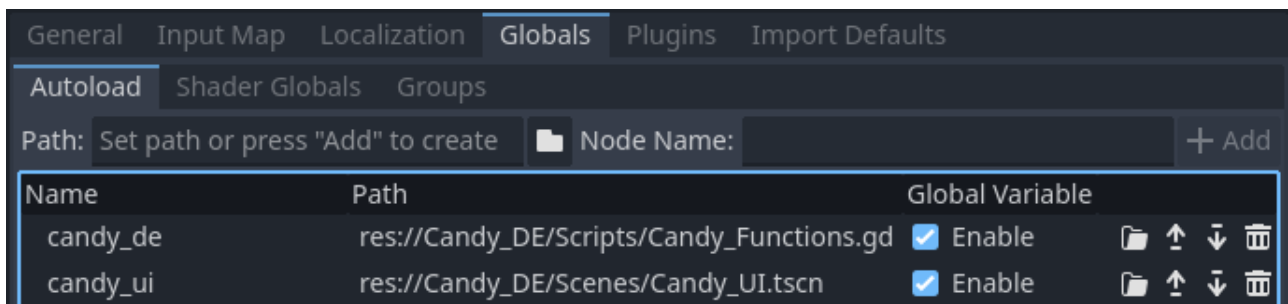
1. Open Godot and open your project.

2. In the unpacked **Candy-Dialogue-Engine-[version]** folder, copy the 'Candy_DE' folder.
3. Paste 'Candy_DE' and to your Godot project folder.
 - **Make sure you place 'Candy_DE' in your project's root (main) folder.**
 - **Do not place it in 'addons': Candy Dialogue Engine is an asset, not a plugin.**
 - The root folder has a file named '**project.godot**'.
4. Return to Godot, it will import the new files automatically.
5. You will see UID errors in the Output console. This is normal: ignore them for now, we will fix it at the end of the installation process.

Notes:

The UID errors happen because the Candy DE files were not created in your project. Since 4.4, Godot assigns unique IDs to all files. Don't worry: Godot has a built-in tool that can fix this in one click. The errors shouldn't happen in older versions of Godot.

STEP 3 - Add Autoloads



1. In Godot, at the top, click **Project**, then **Project Settings**.
2. In the Project Settings window, Click the **Globals** tab, then the **Autoload** sub-tab.
3. Add the 'candy_de' autoload:
 - 1) At the top, next to the "Path" field, click the folder icon to select a file.
 - 2) Select Candy_DE/Scripts/User_Data/Candy_Functions.gd
 - 3) In the "Node Name" field, write 'candy_de'.
 - 4) Click the "+ Add" button.
4. Add the 'candy_ui' autoload:
 - 1) At the top, next to the "Path" field, click the folder icon to select a file.
 - 2) Select Candy_DE/Scenes/Candy_UI.tscn
 - 3) In the "Node Name" field, write 'candy_ui'.

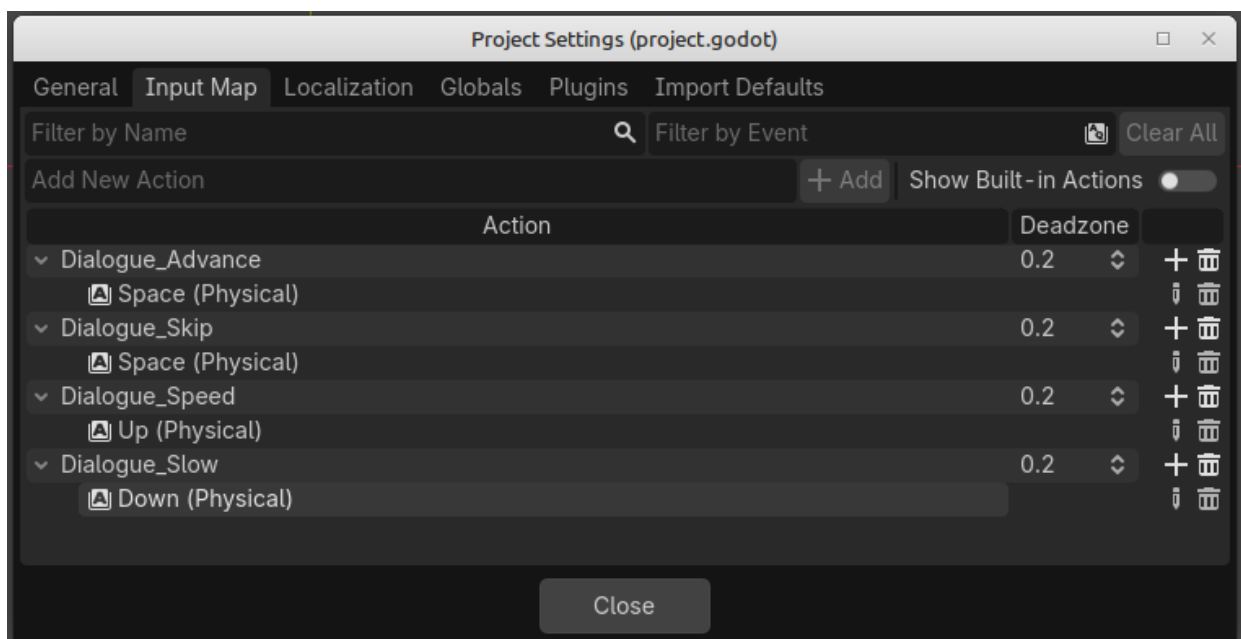
- 4) Click the "+ Add" button.
5. Make sure both autoloads have 'Enable' checked.
6. Do not close the Project Settings window yet.

Notes:

Autoloads (also called 'Globals' or 'Singletons') are scripts or scenes that are always loaded in the background. You can refer to them in your code at anytime by typing their names. For example, to call a function inside the 'candy_de' autoload, you would write 'candy_de.your_function()'.

The candy_ui autoload is technically optional: we **strongly** recommend adding it and using it as the default dialogue engine instance/dialogue UI until you are more familiar with Candy Dialogue Engine. If you don't add it, you must edit Testing_Scene.gd in Candy_DE/Scripts or you will get errors.

STEP 4 - Add Input Actions



1. In the Project Settings window, click the **Input Map** tab.
2. Add the **Dialogue_Advance** input action:
 - 1) In the field that says 'Add New Action', write 'Dialogue_Advance' (no quotes).
 - 2) Click the "+ Add" button.
 - 3) Don't assign input events (keys or buttons) to it yet.
3. You can now repeat this process for three optional actions if you want:
 - 1) **Dialogue_Skip**
 - 2) **Dialogue_Speed**

3) Dialogue_Slow

4. Read the notes below, and use this information to add events (keys/buttons) to these actions.

Notes:

By default, when a character's speech has finished displaying on screen, dialogue will pause until the **Dialogue_Advance** key or button is pressed by the player.

In the engine settings, you can also make dialogue automatically advance after some time, in which case you can also disable manual advancing.

Candy Dialogue Engine also provides an optional typewriter effect: it writes text one letter at a time instead of displaying it all at once. The three optional inputs are only used if the typewriter effect is enabled:

- Dialogue_Skip will make the text display instantly (skip the typewriter effect).
- Dialogue_Speed will make the text write faster (configurable speed).
- Dialogue_Slow will make the text write slower (configurable speed).

These three optional actions must have different keys or buttons, otherwise they will conflict.

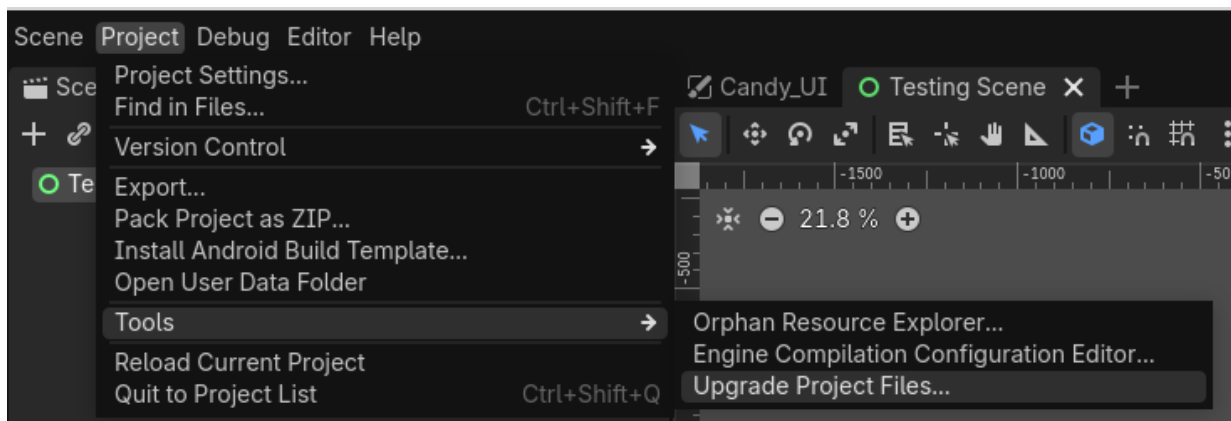
However, the three optional actions can share the same keys and buttons with Dialogue_Advance. This is recommended for player convenience (e.g. press Space to write faster, then press Space again to advance to the next line).

You can assign multiple keys or buttons to Dialogue_Advance (or any other input action).

If you don't want to let players manually advance, you can just not assign a key/button to Dialogue_Advance. However, the Dialogue_Advance key must always be added to the Input Map, or it will cause errors.

Step 5 - Fixing UIDs

This is not required for Godot 4.0 to 4.3. UIDs were introduced in 4.4.



1. Close the Project Settings window.
2. At the top of the main Godot Editor window, click **Project** again, then go to **Tools** and select "**Upgrade Project Files...**"

3. If you are asked to save, do so.
4. When you are asked to restart Godot, restart.

Notes:

When Godot restarts, you might see the UID errors again. These are the error prints from earlier the restart, because Godot doesn't clear the Output when restarting.

To make sure the fix worked, close Godot, then re-open it.

Known Glitch:

In the Godot Editor, while using Candy Dialogue Engine, you may sometimes see an error about a **SubViewport**, related to either speech bubbles or bark bubbles...

This glitch seems completely harmless.

We're not sure what's causing this and it doesn't happen consistently, but it doesn't seem to break anything. You can safely ignore the warning if you ever see it, but please let us know if you notice problems or if you have any helpful information on this.

Next Steps:

The next steps are learning the basics of using Candy DE. Luckily, we provide a lot of documentation about all the features.

To do that, read **Getting Started.pdf**, located in the same unpacked Candy-Dialogue-Engine-[version] you found this document in.

Continue reading below for important information.

Final Notes:

Read this before closing this document, it contains important information.

Why don't we automate the installation process?

Because you should know what Candy Dialogue Engine does in your project. This is important if you ever want to remove it, or if you want to write code that interacts with it: there is nothing mysterious about it this way.

How can you remove Candy Dialogue Engine from your project?

Easy: it's just the installation process in reverse!

Delete the autoloads and the input actions from your Project Settings, then delete the Candy_DE folder (make sure to save any personal files you placed inside of the Candy_DE folder first).

How easy is it to use?

Before you start, there's a few things to understand:

- Candy Dialogue Engine can't make a game for you: you need to have the programming knowledge to make the game you are trying to make.
- You will need at least *some* knowledge of Godot: If you want to make custom UIs, you'll need to know how to make UIs with control nodes in Godot. If you want to use hook functions, you'll need to know how to code what you want those functions to do. If you want to modify variables, you'll need to know how variables work. And so on...
- Candy Dialogue Engine only does some of the work for you (although in some cases, like basic visual novels, that can actually be *all* the work you need).
- Aside from that, Candy Dialogue Engine is easy to use for simple dialogues. You don't have to learn all the features, only the ones you actually use. And creating dialogues is simple.

But what is it really like if you just want to run dialogues in your game?

Basically, just create a dialogue, then make your game call the `start_dialogue()` function (see the documentation for details).

Most features only require either of two things: learning how to use specific dialogue commands, or learning how to configure dialogue settings.

If you're a beginner, it won't teach you GDScript but it can help you learn to think like a programmer: dialogue commands are instructions that run in sequence - "Show X, then Y; if A: do Z; when B happens, change this variable..."

How is the performance?

We don't have actual benchmarks, but dialogues are processed one line after the other, and each line is very quick to process.

The simple answer: you shouldn't notice any lag in normal or even in more advanced cases.

Of course, if you make dialogues play dozens of videos at once, you'll probably notice a performance drop, but that's not specific to Candy Dialogue Engine.

If you use dialogues ***as a programming layer***, then that will be a lot less efficient than GDScript: each dialogue command runs multiple lines of GDScript code to perform its task. It will probably perform faster than you might expect, but logic programmed directly in GDScript will always be faster than the same logic programmed through dialogues.

To mitigate this, you have the option of using a hybrid approach: write functions in GDScript, then call them from dialogues with the §Call command.

To hit performance limits, you would probably have to be programming something complex and very advanced through dialogues - more of an actual program, not standard dialogue use-cases like synchronizing spoken text and media playback.

Creating Dialogues:

In order to create dialogues, you will need the Candy Dialogue Creator standalone application.

You can get it from [GitHub](#): it's completely free and it's published under the MIT License.

- **Candy Dialogue Engine** is the code that runs dialogues in your project. It's not a plugin, so it doesn't have a built-in UI for editing dialogues.
- **Candy Dialogue Creator** is a standalone software, made in Godot, which provides a UI and useful features for creating and editing dialogues. You can then export dialogues from Candy Dialogue Creator to your project. You can download it as a compiled executable for Linux, Windows and MacOS, or as a raw (uncompiled) Godot project.

Documentation is provided to learn how to use Candy Dialogue Engine:

Documentation can be found in the 'Documents' folder (in the same place where you found the 'Candy_DE' folder). In there, you will find a "Guides" folder with many PDF guides about all the features.

We also have a [Youtube](#) channel with tutorial videos if you prefer a more visual explanation.

Please note that at this time, the documentation and videos are not fully updated to version 1.1. This shouldn't be a problem: the differences are mainly just the interface (UI) of Candy Dialogue Creator.

Candy Dialogue Engine is free for non-commercial projects.

If your game doesn't earn any money, you don't need to pay us: thank you for making free games that everyone can enjoy!

Commercial projects require that you purchase a commercial license or pay royalties on your project's revenue.

We offer very good deals for small indie developers: the price is on the higher end of hobbyist/small indie add-ons, but you only need to pay once to make unlimited indie projects. Even if you only use Candy Dialogue Engine for one project, you get a lot of value considering what it helps you do and the time it saves you.

Candy Dialogue Engine requires a lot of work to create and maintain: something of this scale can't be made by working on it a few hours every week-end. This pricing model allows us to work on Candy Dialogue Engine full time, add features and improvements, and develop more assets.

See our [website](#) for more information about prices, license terms and where to buy.

Please report bugs and issues!

This is important. Candy Dialogue Engine is a very extensive program, and we can't test every possible edge case by ourselves. Thankfully, Candy Dialogue Engine is built on a solid, modular core structure, so bugs are always superficial and their causes easy to trace.

This means we can fix bugs and solve your problems very quickly if you report them on the [Candy DE](#) and [Candy DC](#) repositories.

We're here to help:

If you have any questions or need tips and advice on using Candy Dialogue Engine, we'll gladly help you out.

Whether you're encountering technical issues, you don't understand the documentation, or you'd like to use some features for more advanced purposes, just make a post on our [forum](#)!

Feedback is appreciated:

We're always happy to know that people are using our assets - after all, that's why we made them. But more importantly, feedback also helps us improve our assets by telling us what features our users need.

The feedback we seek isn't limited to Candy Dialogue Engine's performance: you can also let us know if the documentation is unclear in some places, or if we can somehow improve our tutorial videos.

You can provide feedback on our [forum](#), on our [Youtube](#) videos, or even by contacting us through our [website](#).

We can showcase your projects:

If you make a game with Candy Dialogue Engine or any of our assets, we will be glad to [showcase](#) it on our website and give you some exposure.

This is offer available for all games made with our assets, including commercial and non-commercial.

See the following links for more information:

[Our website](#): For information, links, etc.

[GitHub](#): To download Candy DE and Candy DC or other assets, and report bugs and issues.

[Our forum](#): For news, sharing feedback, asking for help and advice, requesting features, etc.

[Youtube](#): Mostly for video tutorials on using our assets. We might post teasers, trailers and news sometimes.

[Gumroad](#): For buying commercial licenses. We'll offer more options in the future, such as the official Godot Asset Store once it is complete and accepts payments.

Reminder: Don't forget to read the 'Getting Started.pdf' document provided with your download of Candy DE!